

2.1. Probleme de ressources et critique (Exclusion Mutuelle)

1. Parallélisme, pseudo-parallélisme

On appelle parallélisme l'activation de plusieurs processus au même temps. Si chaque processus s'exécute dans un processeur physique propre à lui, on parle de **vrai parallélisme** (parallélisme réel). On parle de **pseudo-parallélisme** dans le cas où plusieurs processus commutent sur un seul processeur. Le parallélisme est obtenu par commutation temporelle d'un processus à l'autre sur le processeur. L'utilisateur a l'illusion d'un parallélisme réel. On l'appelle aussi **concurrency**. Le pseudo-parallélisme est un moyen de simuler le parallélisme réel sur un processus unique.

2- Relations entre processus :

Les relations entre processus peuvent prendre deux formes :

- *Compétition* pour l'usage d'une ressource partagée : L'exemple le plus courant de compétition est celui de l'allocation de processeur. Une politique d'allocation attribue le processeur, par tranches de temps successives, aux processus qui en ont besoin.
- *Coopération* pour l'exécution coordonnée d'une tâche commune : Un exemple simple de coopération est celui de la transmission de messages entre deux processus à travers une zone de mémoire commune

3. Problème de ressource critique (Exclusion Mutuelle)

Dans un système d'exploitation multiprogrammé en temps partagé, plusieurs processus s'exécutent en pseudo-parallèle ou en parallèle et partagent des objets (mémoires, imprimantes, etc)..

Le partage d'objets sans précaution particulière peut conduire à des résultats imprévisibles. L'état final des données dépend de l'ordonnancement des processus. Lorsqu'un processus modifie un objet partagé, les autres processus ne doivent ni la lire, ni la modifier jusqu'à ce que le processus ait terminé de la modifier. Autrement dit, l'accès à l'objet doit se faire en exclusion mutuelle.

Exemple

Considérons deux clients A et B d'une banque qui partagent un même compte. Le solde du compte est 1000DA. Supposons qu'en même temps, les deux clients lancent chacun une opération. Le client A demande un dépôt de 2000DA alors que le client B veut faire un dépôt de 500DA. Si la requête de A est exécutée par un processus P et la requête de B est exécutée par un processus Q, nous aurons les séquences d'exécution suivantes :

Processus P:	Processus Q:
<pre>{ (p1) A=Solde; /*lire solde*/ (p2) A=A+2000; (p3) Solde=A; }</pre>	<pre>{ (Q1) A=Solde; /*lire solde*/ (Q2) A=A+500; (Q3) Solde=A; }</pre>

Si les processus s'exécutent en séquentielle, c'est à dire, ni en parallèle ni en pseudo-parallèle, le résultat final du compte sera: $1000+2000+500=3500$.

si les processus s'exécutent en concurrence plusieurs cas sont possibles:

Si: P1Q1Q2Q3P2P3, Alors le résultat sera: **3000** . Donc erreur!

Si: Q1Q2P1P2P3Q3, Alors le résultat sera: **1500** . Donc erreur!

4. Sections critiques

Il faut empêcher l'utilisation simultanée de la variable commune solde. Lorsqu'un thread (ou un processus) exécute la séquence d'instructions (1, 2 et 3), l'autre doit attendre jusqu'à ce que le premier ait terminé cette séquence. On appelle **section critique**, la suite d'instructions qui opère sur un ou plusieurs objets partagés et qui nécessitent une utilisation exclusive des objets partagés. Chaque thread (ou processus) a ses propres sections critiques. Les sections critiques des différents processus ou threads qui opèrent sur des objets communs doivent s'exécuter en exclusion mutuelle. Avant d'entamer l'exécution d'une de ses sections critiques, un thread (ou un processus) doit s'assurer de l'utilisation exclusive des objets partagés manipulés par la section critique.

5. Solution du problème

Quatre conditions sont nécessaires pour réaliser correctement une exclusion mutuelle :

1. Deux processus ne peuvent être en même temps dans leurs sections critiques.
2. Aucune hypothèse ne doit être faite sur les vitesses relatives des processus et sur le nombre de processeurs.
3. Aucun processus suspendu en dehors de sa section critique ne doit bloquer les autres processus.
4. Aucun processus ne doit attendre trop longtemps avant d'entrer en section critique (attente bornée).

Pour résoudre le problème d'exclusion mutuelle, on doit encadrer chaque section critique SC par des opérations spéciales qui visent à assurer l'utilisation exclusive des objets partagés.

Schema de la solution:

```
<Demande d'entrée en SC>;  
[<SECTION_CRITIQUE>]  
<Sortie de SC>;
```

Pour le compte bancaire la solution sera comme suit:

Processus P:	Processus Q:
<pre>{ Demande d'entrée en SC; (p1) A=Solde; /*lire solde*/ (p2) A=A+2000; (p3) Solde=A; <Sortie de SC>; }</pre>	<pre>{ Demande d'entrée en SC; (Q1) A=Solde; /*lire solde*/ (Q2) A=A+500; (Q3) Solde=A; <Sortie de SC>; }</pre>

5.1 Solution 1 : Masquage des interruptions

Avant d'entrer dans une section critique, le processus masque les interruptions. Il les restaure à la fin de la section critique. Il ne peut être alors suspendu durant l'exécution de la section critique.

Problèmes :

- Solution dangereuse : si le processus, pour une raison ou pour une autre, ne restaure pas les interruptions à la sortie de la section critique, ce serait la fin du système.
- Elle n'assure pas l'exclusion mutuelle, si le système n'est pas monoprocesseur (le masquage des interruptions concerne uniquement le processeur qui a demandé l'interdiction). Les autres processus exécutés par un autre processeur pourront donc accéder aux objets partagés.

En revanche, cette technique est parfois utilisée par le système d'exploitation.

5.2 Slution 2: Attente active et verrouillage

Utiliser une variable partagée verrou, unique, initialisée à 0. Pour rentrer en section critique, un processus doit tester la valeur du verrou.

- Si verrou = 0, le processus met le verrou à 1 puis exécute sa section critique. A la fin de la section critique , il remet le verrou à 0.
- Sinon, il attend (attente active) que le verrou devienne égal à 0 : while(verrou !=0) ;

int verrou =0; /*une variable partagée*/	
Processus P:	Processus Q:
{	{
while(verrou !=0);	while(verrou !=0);
verrou=1;	verrou=1;
A=Solde; /*lire solde*/	A=Solde; /*lire solde*/
A=A+2000;	A=A+500;
Solde=A;	Solde=A;
verrou=0;	verrou=0;
}	}

Problème: La solution est érronée, elle n'assure pas l'exclusion mutuelle :

Supposons qu'un processus est suspendu suite à une interruption horloge juste après avoir lu la valeur du verrou qui est égal à 0. Ensuite, un autre processus prend le processeur. Ce dernier teste le verrou qui est toujours égal à 0, met le verrou à 1 et entre dans sa section critique. Ce processus est suspendu avant de quitter la section critique. Le premier processus est alors réactivé, il entre dans sa section critique et met le verrou à 1. Alors les deux processus sont en même temps en section critique!

5.3 Solution 3 : Attente active avec alternance

Utiliser une variable tour qui mémorise le tour du processus qui doit entrer en section critique. tour est initialisée à 0.

int tour =0; /*une variable partagée*/	
Processus P:	Processus Q:
{	{
while(tour !=0);	while(tour !=1);
A=Solde;	A=Solde;
A=A+2000;	A=A+500;
Solde=A;	Solde=A;
tour=1;	tour=0;
}	}

Problèmes :

Un processus peut être bloqué par un processus qui n'est pas en section critique.

- P lit la valeur de tour qui vaut 0 et entre dans sa section critique. Il est suspendu et Q est exécuté. Q teste la valeur de tour qui est toujours égale à 0. Il entre donc dans une boucle en attendant que tour prenne la valeur 1. Il est suspendu et P est élu de nouveau. P quitte sa

section critique, met tour à 1 et entame sa section non critique. Il est suspendu et Q est exécuté. Q exécute rapidement sa section critique, tour = 0 et sa section non critique. Il teste tour qui vaut 0. Il attend que tour prenne la valeur 1 .

- Cette solution utilise aussi l'attente active consomme du temps CPU.

5.4 Solution de Dekker pour deux processus:

Chacun des deux processus fait connaître (à l'autre) sa candidature. En cas de candidatures simultanées (collision), le conflit est réglé en donnant priorité à un processus. Pour avoir une solution équitable et respecter l'ancienneté des requêtes, la priorité doit être variable

<pre> int Premier= 1; bool Candidat_P=false, Candidat_Q=false; </pre>	
Processus P:	Processus Q:
<pre> { Candidat_P = true; while (Candidat_Q) { while (Premier == 2)Candidat_P = false; Candidat_P = True; } A=Solde; A=A+2000; Solde=A; Candidat_P= False; Premier = 2; } </pre>	<pre> { Candidat_Q = true; while (Candidat_P) { while (Premier == 1)Candidat_Q = false; Candidat_Q= True; } A=Solde; /*lire solde*/ A=A+500; Solde=A; Candidat_P= false; Premier = 1; } </pre>

Problème : attente active = consommation du temps CPU

5.5 Solution de Peterson pour deux processus:

Après avoir fait connaître sa candidature, chaque processus se propose comme dernier. En cas de conflit, ceci bloque le demandeur jusqu'à l'arrivée d'un nouveau dernier. C'est donc celui qui se propose en dernier comme dernier qui est la victime retenue.

<pre> int Dernier= 1; bool Candidat_P=false, Candidat_Q=false; </pre>	
Processus P:	Processus Q:
<pre> { Candidat_P = True; Dernier = 1 ; while (Candidat_Q and Dernier==1)do; /* LA SECTION CRITIQUE */ Candidat_P = false; } </pre>	<pre> { Candidat_Q = True; Dernier = 2 ; while (Candidat_P and Dernier ==2)do; /* LA SECTION CRITIQUE */ Candidat_Q = false; } </pre>

Problème : attente active = consommation du temps CPU.

Note: Les solutions de Peterson et de Dekker peuvent être étendues pour un nombre N de processus.

5.6 Solution matérielle: L'instruction Test and Set Lock (TSL)

L'instruction Test and Set Lock « int TSL (int b) » exécute de manière indivisible les opérations suivantes:

- récupère la valeur de b,
- affecte la valeur 1 à b et
- retourne l'ancienne valeur de b.

Lorsqu'un processeur exécute l'instruction TSL, il verrouille le bus de données pour empêcher les autres processeurs d'accéder à la mémoire pendant la durée de l'exécution. Cette instruction peut être utilisée pour établir et supprimer des verrous (assurer l'exclusion mutuelle).

Solution:

<pre>int verrou= 0; bool Candidat_P=false, Candidat_Q=false;</pre>	
<u>Processus P:</u>	<u>Processus Q:</u>
<pre>{ while(TSL(verrou)) ; /* LA SECTION CRITIQUE */ verrou=0; }</pre>	<pre>{ while(TSL(verrou)) ; /* LA SECTION CRITIQUE */ verrou=0; }</pre>

Problèmes de TSL :

Elle utilise l'attente active = consommation du temps CPU .

Note: TSL la slution TSL présenté ci-dessus est valide pour n'importe quel nombre de processus.

Une autre variante de : L'instruction swap

Plusieurs architectures supportent également l'instruction swap qui permet de permuter les valeurs de deux variables exécute de manière indivisible.

swap (a, b);

est équivalent à :

```
temp = a;
a = b;
b = temp;
```